A MINI PROJECT REPORT ON

TIME COMPRESSION ENGINE (TCE): NARRATIVE-PRESERVING VIDEO INTELLIGENCE, MULTI-ACTOR RE-IDENTIFICATION, AND TEMPORAL SEMANTIC COMPRESSION FOR 24-HOUR SURVEILLANCE FOOTAGE

SUBMITTED IN PARTIAL FULFILMENT OF THE REQUIREMENTS
FOR THE DEGREE OF
BACHELOR OF TECHNOLOGY IN COMPUTER ENGINEERING
(Academic Year 2026–27)

BY

Dhruvin Shah (UID: 2024300224)
Kavya Jagani (UID: 2024300091)
Deven Thakkar (UID: 2024300261)

UNDER THE GUIDANCE OF
Prof. Swapnali Kurhade
Department of Computer Engineering

DEPARTMENT OF COMPUTER ENGINEERING
SARDAR PATEL INSTITUTE OF TECHNOLOGY (SPIT)
(Autonomous Institute Affiliated to University of Mumbai)
Munshi Nagar, Andheri (West), Mumbai – 400058
ACADEMIC YEAR: 2026–27

CERTIFICATE OF APPROVAL

This is to certify that the Mini Project entitled "Time Compression Engine (TCE): Narrative-Preserving Video Intelligence, Multi-Actor Re-Identification, and Temporal Semantic Compression for 24-Hour Surveillance Footage" is a bona fide record of authentic engineering work carried out by:

1. **Dhruvin Shah (UID: 2024300224)**
2. **Kavya Jagani (UID: 2024300091)**
3. **Deven Thakkar (UID: 2024300261)**

in partial fulfilment of the requirements for the award of the Degree of Bachelor of Technology in Computer Engineering (Third Year, Semester V/VI) during the academic year 2026–27 at Sardar Patel Institute of Technology, Mumbai.

_____	_____
Prof. Swapnali Kurhade	Dr. / Prof. Head of Department
Project Guide	Department of Computer Engineering
Department of Computer Engineering	S.P.I.T., Mumbai
S.P.I.T.,	Mumbai
_____	_____
Internal Examiner	External Examiner
Date:	Date:

DECLARATION

We hereby declare that this written submission represents our original ideas and engineering work in our own words. Where others' ideas, research publications, software libraries, mathematical formulas, or architectural patterns have been referenced, we have adequately cited and acknowledged the original sources in accordance with academic standards.

We confirm that this project report adheres strictly to the APNI Leap model prescribed by Sardar Patel Institute of Technology, distinguishing between proposed architecture, implemented code, and experimentally measured benchmarks on genuine historical surveillance video datasets.

We further confirm that this document contains truthful representations of system performance, verified test cases, and authentic execution telemetry. The group has independently maintained the Git repository with regular commits and an organized development history.

Date: 24th October 2026
Place: Mumbai

1. _____
   Dhruvin Shah (UID: 2024300224)

2. _____
   Kavya Jagani (UID: 2024300091)

3. _____
   Deven Thakkar (UID: 2024300261)

ACKNOWLEDGEMENT

We express our profound gratitude to our project guide, Prof. Swapnali Kurhade, for her valuable mentorship, technical guidance, and constructive critique throughout the design, development, and testing phases of the Time Compression Engine. Her insightful recommendations regarding directed narrative graph construction, multi-modal confidence fusion, and temporal budget optimization significantly elevated the analytical rigor of this work.

We also extend our sincere appreciation to the Head of the Department of Computer Engineering and the administration of Sardar Patel Institute of Technology for providing state-of-the-art computational infrastructure, high-performance GPU development environments, and access to authoritative academic research databases.

We are grateful to our laboratory instructors and department faculty members who reviewed our design milestones during the intermediate project reviews. Finally, we thank our peers and families for their continuous patience, encouragement, and understanding during the intensive development lifecycle of this project.

Dhruvin Shah (UID: 2024300224)
Kavya Jagani (UID: 2024300091)
Deven Thakkar (UID: 2024300261)
(Department of Computer Engineering, Sardar Patel Institute of Technology)

ABSTRACT

Closed-Circuit Television (CCTV) infrastructure records terabytes of continuous video daily, yet human cognitive limits prevent effective 24-hour review. Traditional video codecs (H.264, H.265) achieve pixel-level compression but do not reduce video duration, while naive motion-triggered summarizers produce disorienting, fragmented bursts that destroy evidentiary context. This project introduces the Time Compression Engine (TCE v1.1), a patent-grade, narrative-preserving video intelligence platform that converts multi-hour surveillance footage into structured, causally continuous narrative graphs, multi-actor identity dossiers, and executive investigative digests.

The architecture implements a decoupled 3-engine, 12-stage sequential pipeline: (1) Perception Engine for adaptive MOG2 frame sampling, scene cut detection, YOLO11 object detection, and Kalman tracking; (2) Semantic Engine for kinematic locomotion recognition, localized open-vocabulary scene verification (suppressing 100% of perspective-drift stair climbing false alarms), and weighted confidence fusion; and (3) Temporal Engine for directed narrative graph construction, multi-factor ranking, and an atomic compression policy that preserves entire causal chains (Entry -> Actions -> Exit).

Implemented with a FastAPI backend and a Next.js 14 reactive web dashboard, TCE features continuous MobileNetV3 appearance-metric ReID across 6 ground-truth occupants, deep desktop equipment inventorying, synchronized canvas HUD overlays, and dynamic natural language scene brief synthesis. Benchmarked on real surveillance feeds, TCE achieves an empirical Real-Time Factor (RTF) of 0.199 (5.03x real-time throughput) with a peak VRAM footprint of 983 MB on an NVIDIA RTX 5050 GPU, preserving 100% of narrative chains under a 40% compression target.

Index Terms— *Keywords: Video Intelligence, Narrative Preservation, Temporal Compression, Object Tracking, Re-Identification, Computer Vision, Surveillance Analytics.*

LIST OF ABBREVIATIONS & SYMBOLS

Abbreviation	Full Expansion
API	Application Programming Interface
CCTV	Closed-Circuit Television
COCO	Common Objects in Context (80-Class Benchmark)
CUDA	Compute Unified Device Architecture (NVIDIA)
DAG	Directed Acyclic Graph
DAW	Digital Audio Workstation (Sequencer Track Layout)
DFD	Data Flow Diagram
FFmpeg	Fast Forward Moving Picture Experts Group Multimedia Suite
HUD	Heads-Up Display
IoP	Intersection-over-Person Spatial Metric
IoU	Intersection-over-Union (Jaccard Index)
JSON	JavaScript Object Notation
KPI	Key Performance Indicator
MOG2	Mixture of Gaussians v2 Adaptive Background Subtractor
NMS	Non-Maximum Suppression
ORM	Object-Relational Mapping (SQLAlchemy)
ReID	Person Re-Identification Deep Feature Embeddings
ROI	Region of Interest
RTF	Real-Time Factor (Execution Time / Video Duration)
SAHI	Slicing Aided Hyper Inference
SRS	Software Requirements Specification
TCE	Time Compression Engine
UML	Unified Modeling Language
VMS	Video Management Software
VRAM	Video Random Access Memory (GPU Dedicated)
YOLO	You Only Look Once (State-of-the-Art Object Detector)

TABLE OF CONTENTS

Section / Chapter Title	Page Number
Certificate of Approval	2
Declaration	3
Acknowledgement	4
Abstract & Keywords	5
List of Abbreviations & Symbols	6
Table of Contents	7
Chapter 1: Introduction and Problem Definition	9
1.1 Project Background and Industrial Context	9
1.2 Problem Statement	9
1.3 Motivation & Engineering Objectives	10
1.4 Scope of the Project	10
1.5 Novelty and Originality	11
1.6 Success Criteria and Key Performance Indicators (KPIs)	11
Chapter 2: Literature Review and Study of Existing Systems	13
2.1 Study of Commercial Surveillance Systems	13
2.2 Review of Research Papers and Academic Models	13
2.3 Comprehensive Comparative Analysis	14
2.4 Identified Research & Engineering Gap	14
Chapter 3: Software Requirements Specification (SRS)	15
3.1 Product Perspective & Intended Stakeholders	15
3.2 Detailed Functional Requirements (FR-01 to FR-12)	15
3.3 Non-Functional Requirements & Engineering Standards	16
Chapter 4: System Architecture and Design	17
4.1 Decoupled 3-Engine Architecture	17
4.2 Database Design & Relational Schema	17
Chapter 5: Algorithm Design and Data Structures	19
5.1 Mathematical Formulations of Key Pipeline Algorithms	19
5.2 Time & Space Complexity Analysis	19
Chapter 6: Subject-Wise Engineering Concepts & APNI Leap Alignment	21
Chapter 7: Implementation and Module-Wise Coding	22
7.1 Technical Stack & Development Environment	22
7.2 Front-End & Back-End Implementation Highlights	22
Chapter 8: Scalability, Performance and Resource Management	23
8.1 Workload Model & Empirical Resource Utilization	23
8.2 Performance Results & Real-Time Factor (RTF)	23
Chapter 9: Testing and Validation	24
9.1 Automated Test Suite & Coverage (693 Tests)	24

9.2 Critical Defect Resolution	24
Chapter 10: Results, Demonstration and Discussion	25
10.1 Ground-Truth Verification on Surveillance Video	25
10.2 System Demonstration Screenshots (Figures 10.1 - 10.7)	26
Chapter 11: Conclusion and Future Scope	30
11.1 Conclusion	30
11.2 Major Engineering Contributions	30
11.3 Limitations and Future Work	30
References & Bibliography	31

CHAPTER 1: INTRODUCTION AND PROBLEM DEFINITION

1.1 Project Background and Industrial Context

Surveillance cameras represent the single largest producer of digital visual data in the world today. Modern enterprise environments—including corporate campuses, research laboratories, manufacturing plants, financial data centers, and multi-modal transit hubs—maintain continuous video recording infrastructure consisting of dozens or hundreds of high-definition network IP cameras. These devices operate 24 hours a day, 7 days a week, capturing high-resolution 1080p and 4K streams at framerates ranging from 15 to 30 frames per second.

However, video acquisition capability has completely outstripped human cognitive capacity. The fundamental limitation in modern physical security is not a lack of video data, but an acute shortage of human attention to review it. In standard enterprise operations, surveillance video is passively recorded to Network Video Recorders (NVRs) or cloud storage buckets, retained for a regulatory period (typically 30 to 90 days), and overwritten without ever having been viewed by human eyes unless an incident is formally reported.

When an incident does occur—such as commercial theft, intellectual property leakage, unauthorized access, or safety violations—forensic investigators are tasked with identifying critical events across hours or days of footage. The prevailing investigative methodology remains manual, fast-forwarded video scrub at 4x, 8x, or 16x playback speeds. Extensive human-factors research has demonstrated that an observer's ability to detect subtle, critical security events drops by more than 50% after just 20 minutes of continuous screen monitoring. The result is delayed incident resolution, high human labor expenditure, and a high probability that subtle precursor events remain undiscovered.

1.2 Problem Statement

Modern commercial and open-source video analysis solutions fail forensic teams in three structural ways:

1. Blind Pixel-Level Compression: Standard pixel codecs compress storage space but do nothing to reduce viewing duration. A 24-hour video compressed with H.265 still requires 24 hours of linear human playback.

2. Destruction of Causal Continuity & Narrative Chains: Commercial video summarizers and motion-triggered snapshot generators extract high-motion fragments while discarding quiet periods. In doing so, they sever the causal linkage between events. An investigator sees an actor

standing near an open vault, but the antecedent actions (how they bypassed the perimeter, whether an accomplice staged the access card) and subsequent actions (where the individual departed, what assets were carried) are completely excised.

3. Absence of Multi-Actor Re-Identification & Semantics: Existing systems produce isolated bounding boxes or disconnected frame logs. They lack the intelligence to link multiple appearances of the same individual into persistent actor dossiers, infer higher-level human activities, or generate cohesive natural language investigative briefs.

1.3 Motivation & Engineering Objectives

The central motivation behind the Time Compression Engine is to build an autonomous video intelligence system that compresses time while rigorously preserving evidentiary narrative integrity. Rather than producing a disjointed compilation of motion bursts, TCE treats surveillance video as a directed chronological story.

Objective 1 (Perception Intelligence): To engineer a fully automated 12-stage pipeline that ingests raw surveillance video, extracts high-utility keyframes via adaptive background subtraction, performs YOLO11-based multi-class detection, and tracks targets via Kalman-filtered state estimators.

Objective 2 (Semantic & Identity Fusion): To recognize complex actor behaviors through kinematic velocity analysis, verify subtle scene interactions via localized open-vocabulary semantic models, and maintain actor identity continuity using deep appearance metric ReID across camera occlusions.

Objective 3 (Narrative Preservation & Temporal Compression): To construct directed narrative graphs connecting actor arrival, actions, object interactions, and departure, applying a multi-factor ranking policy that selects high-salience chains within a target time budget.

Objective 4 (Forensic User Experience): To provide enterprise security personnel with an interactive Cyberpunk HUD forensic dashboard featuring synchronized canvas overlays, timeline scrubbing, actor dossiers, and dynamic narrative summaries.

1.4 Scope of the Project

The operational scope of the Time Compression Engine encompasses corporate offices, laboratory facilities, commercial warehouses, data centers, and institutional campuses equipped with fixed-viewpoint high-definition CCTV cameras. The platform supports continuous streams

and pre-recorded archival files in standard container formats (MP4, MKV, AVI, MOV) encoded with H.264/AVC or H.265/HEVC.

While engineered for single-camera multi-hour scene analysis in its v1.1 release, the underlying data architecture—specifically the Relational Schema, Actor Dossier representations, and MobileNetV3 512-dimensional embedding spaces—is explicitly designed for horizontal extension across multi-camera enterprise deployments.

1.5 Novelty and Originality

The Time Compression Engine introduces three key technical innovations:

1. Narrative Chain Atomicity: Conventional summarizers evaluate video segments in isolation. TCE enforces atomic preservation of entire narrative chains. If an event is selected for inclusion in the summary, its complete causal chain (Entry -> Antecedent Actions -> Core Event -> Consequent Actions -> Exit) is guaranteed preservation, ensuring evidentiary integrity.

2. Localized Open-Vocabulary Scene Verification: Existing systems suffer high false-positive rates when perspective depth drift causes walking individuals to be misclassified as climbing stairs. TCE introduces a localized 1.5x bounding-box crop mechanism evaluated by open-vocabulary models to verify the physical presence of architectural elements before confirming kinematic alerts, eliminating 100% of perspective drift false alarms.

3. Multi-Factor Confidence Fusion: Unlike static rule engines, TCE computes composite salience scores by fusing kinematic motion, duration weight, identity uniqueness, and spatial centrality using mathematically calibrated sigmoid normalization curves.

1.6 Success Criteria and Key Performance Indicators (KPIs)

The quantitative engineering targets established for system validation are detailed below:

KPI ID	Metric Description	Target Threshold	Measured Value	Status
KPI-01	Real-Time Factor (RTF)	<= 0.35 (>= 2.85x RT)	0.199 (5.03x RT)	PASSED
KPI-02	Narrative Chain Preservation	100 % Causal Chains	100 % Preserved	PASSED
KPI-03	Stair False Alert Suppression	>= 95 % Suppression	100 % (69/69 Suppressed)	PASSED

KPI-04	Identity Consistency across Cuts	>= 90 % Dossier Accuracy	96.4 % Verified	PASSED
KPI-05	Automated Test Suite Pass Rate	100 % (Zero Regressions)	100 % (693/693 Tests)	PASSED
KPI-06	Peak GPU VRAM Utilization	<= 4,096 MB	983 MB Peak	PASSED

CHAPTER 2: LITERATURE REVIEW AND STUDY OF EXISTING SYSTEMS

2.1 Study of Commercial Surveillance Systems

Commercial Video Management Software (VMS) platforms, such as Milestone XProtect, Genetec Omnicast, and Avigilon Control Center, serve as the enterprise standard for physical security. These systems excel at media routing, PTZ control, multi-sensor storage management, and basic motion detection. However, their intelligence features remain fundamentally constrained:

- Traditional VMS platforms rely heavily on pixel change thresholds or basic bounding box detection. In high-traffic environments, they trigger thousands of false motion events daily due to lighting shifts, foliage movement, or shadow variations.
- Video Synopsis solutions superimpose moving objects from different time windows onto a single static background canvas. While this condenses visual display, it destroys the true chronological narrative, creates severe visual clutter during crowded intervals, and confuses forensic investigators regarding which actor arrived first.
- Cloud-based surveillance AI solutions impose recurring subscription costs, introduce severe latency over wide-area networks, and present significant compliance risks regarding private video transmission.

2.2 Review of Research Papers and Academic Models

Academic research in video summarization and surveillance analytics has explored multiple paradigms over the past two decades:

1. Keyframe Extraction & Clustering: Early approaches utilized visual clustering (k-means over color histograms, SIFT, or HOG descriptors) to select keyframes that minimize visual redundancy. While computationally light, these approaches possess zero semantic awareness and fail to capture sequential temporal relationships.

2. Video Synopsis & Trajectory Shifting: Pritch et al. formalized video condensation by formulating object trajectory rearrangement as an energy minimization problem solved via graph cuts. While mathematically elegant, synopsis algorithms remove the temporal ordering of events, making it impossible to determine causality during forensic retrospectives.

3. Tracking-by-Detection & Deep Embedding Models: With the advent of deep single-stage detectors (YOLO, SSD) and association algorithms (SORT, DeepSORT, ByteTrack), tracking actors across continuous frames achieved high accuracy. However, standalone trackers degrade significantly during long occlusions, lack identity persistence across distinct scene cuts, and do not synthesize high-level behavioral narratives.

2.3 Comprehensive Comparative Analysis

A rigorous comparative matrix contrasting TCE against commercial and research paradigms is presented below:

Feature / Criterion	Traditional VMS	Video Synopsis	DeepSORT Alone	TCE v1.1
Temporal Compression	None (1.0x)	High (~10x)	None (1.0x)	Adaptive (2x - 10x)
Causal Continuity	Preserved	Severed / Distorted	Preserved	Strictly Preserved
Multi-Actor Dossiers	No	No	Short-term Only	Persistent (ReID)
Scene Verification	Basic Motion Only	None	None	Localized Open-Vocab
Equipment Tracking	No	No	Bounding Boxes Only	Spatial Relationship Engine
Natural Language Briefs	No	No	No	Automated Narrative Synth
Local GPU Execution	Proprietary Server	Cloud / Expensive	Local Compatible	Local Lightweight (983 MB)

2.4 Identified Research & Engineering Gap

The literature and market review reveals a conspicuous void: no existing system combines adaptive temporal compression with strict causal narrative preservation, localized semantic verification, and persistent identity dossiers within an edge-efficient compute envelope. The Time Compression Engine was engineered specifically to bridge this gap.

CHAPTER 3: SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

3.1 Product Perspective & Intended Stakeholders

The Time Compression Engine is an independent, self-contained video intelligence platform. It interfaces with existing IP camera networks, archival storage repositories, and corporate network file systems via standard HTTP REST APIs. Intended stakeholders include Forensic Security Analysts, Enterprise Facility Managers, Compliance Officers, and Academic Researchers.

3.2 Detailed Functional Requirements

Req ID	Requirement Name	Engineering Description	Verification Method
FR-01	Video Stream Ingestion	Ingest MP4, MKV, AVI, MOV with FFprobe stream metadata extraction.	Automated API Test
FR-02	Adaptive Keyframe Sampling	MOG2 background difference thresholding with dynamic skip (1 to 15 frames).	Benchmark Telemetry
FR-03	Scene Cut Detection	Histogram difference detection across consecutive frames (threshold theta = 0.35).	Visual Unit Test
FR-04	Multi-Class Object Detection	YOLO11 inference targeting person, monitor, laptop, keyboard, backpack.	mAP Benchmark
FR-05	Multi-Object Tracking	Kalman-filtered state estimation with Hungarian association (IoU >= 0.30).	MOTA Validation
FR-06	Actor Re-Identification	Continuous appearance metric extraction (MobileNetV3) with cosine distance matching.	Dossier Cluster Eval
FR-07	Kinematic Event Classification	Detect walking, loitering, running, and rapid trajectory shifts.	Kinematic Rules Suite
FR-08	Localized Scene Verification	SceneStairGuard 1.5x ROI crop evaluation using YOLO-World v2 models.	Stair Dataset Test
FR-09	Directed Narrative Graph	Construct DAG connecting actor arrival, actions, interactions, and exit.	Graph Topology Check

FR-10	Salience Ranking & Knapsack	Multi-factor salience scoring and bounded-budget temporal knapsack selection.	Coverage Metric
FR-11	Lossless Stream Export	FFmpeg selective concat filter pipeline generating compressed H.264 video.	Codec Integrity Check
FR-12	Forensic Web Dashboard	Next.js 14 reactive HUD with Cinema Player, timeline, and dossiers.	E2E UI Test Suite

3.3 Non-Functional Requirements & Engineering Standards

NFR-01 (Throughput & Latency): The system guarantees Real-Time Factor (RTF) <= 0.35 on commodity NVIDIA RTX 5050 GPU hardware (processing a 10-minute video in under 3.5 minutes). Video stream delivery latency across API endpoints is under 150 ms.

NFR-02 (Security & Air-Gapped Privacy): All core processing pipelines operate 100% locally with zero internet dependency, ensuring complete privacy compliance (GDPR, HIPAA, SOC-2).

NFR-03 (Resource Utilization): Runtime host memory is bounded strictly below 4.0 GB RAM, and dedicated GPU VRAM utilization is maintained under 1.5 GB.

CHAPTER 4: SYSTEM ARCHITECTURE AND DESIGN

4.1 Decoupled 3-Engine Architecture

The Time Compression Engine is structured into three logically decoupled, sequentially communicating processing engines encompassing 12 distinct pipeline stages:

1. Perception Intelligence Engine (Stages S01 to S06): Ingests the media stream, extracts frames using adaptive MOG2 background subtraction, identifies shot boundaries, executes YOLO11 object inference, tracks actors across occlusions using Kalman filtering, and extracts deep appearance embeddings for multi-actor ReID.

2. Semantic Understanding Engine (Stages S07 to S09): Translates raw geometric tracking trajectories into structured behavioral semantics. Evaluates kinematic locomotion, applies localized open-vocabulary scene verification (SceneStairGuard), tracks spatial object proximity, and fuses multi-modal confidence scores.

3. Temporal Compression Engine (Stages S10 to S12): Constructs a directed narrative graph (DAG) representing causal event dependencies, executes multi-factor salience ranking, solves the bounded-budget knapsack problem, and orchestrates lossless FFmpeg video assembly.

4.2 Database Design & Relational Schema

System persistence is implemented via SQLite with SQLAlchemy ORM. The relational schema consists of six core entities:

Table Name	Primary Key	Foreign Keys	Key Attributes & Types	Purpose
videos	video_id (VARCHAR)	None	filename, duration, fps, width, height, status	Ingested media catalog
jobs	job_id (VARCHAR)	video_id -> videos	profile, progress, current_stage, created_at	Processing pipeline execution state
tracks	track_id (VARCHAR)	job_id -> jobs	actor_id, start_time, end_time, avg_conf	Kalman-filtered actor trajectories
events	event_id (VARCHAR)	job_id, track_id	event_type, start_time, end_time, confidence	Detected semantic occurrences
story_nodes	node_id	job_id, event_id	salience_score, is_selected,	Narrative graph decision

	(VARCHAR)		causal_depth	vertices
actors	actor_id (VARCHAR)	job_id -> jobs	portrait_path, total_events, first_seen, last_seen	Persistent occupant dossiers

CHAPTER 5: ALGORITHM DESIGN AND DATA STRUCTURES

5.1 Mathematical Formulations of Key Pipeline Algorithms

1. Adaptive Frame Sampling (S02): OpenCV MOG2 background subtraction models the pixel intensity history using a Gaussian mixture model. Let I(x, y, t) be the pixel intensity at time t. The probability of observing the current pixel is: P(x_t) = SUM [w_{i,t} * eta(x_t, mu_{i,t}, Sigma_{i,t})]. When background difference ratio D_diff falls below threshold tau_diff, frame sampling step s scales dynamically up to s_max, skipping redundant static intervals without missing dynamic action.

2. Kalman State Estimation & Hungarian Matching (S05): Actor state is modeled as x = [u, v, gamma, h, u_dot, v_dot, gamma_dot, h_dot]^T. The state transitions via x_k = F * x_{k-1} + w_k, with measurement z_k = H * x_k + v_k. Association between predicted tracks and YOLO11 detections is resolved using the Hungarian algorithm over an Intersection-over-Union (IoU) cost matrix: C_{i,j} = 1 - IoU(box_i, box_j).

3. Multi-Factor Salience Scoring & Knapsack Policy (S11): For each detected event e_i, the composite salience score S(e_i) is computed via multi-factor fusion: S(e_i) = w_m * S_motion(e_i) + w_d * S_duration(e_i) + w_u * S_uniqueness(e_i) + w_c * S_centrality(e_i), where weights satisfy SUM(w) = 1.0. Given total time budget T_target, the compression policy maximizes SUM [S(e_i) * x_i] subject to SUM [duration(Chain(e_i)) * x_i] <= T_target, where x_i in {0, 1}.

5.2 Time & Space Complexity Analysis

Pipeline Stage	Algorithm Applied	Time Complexity	Space Complexity	Empirical Latency
S02 Adaptive Extract	MOG2 Background Diff	O(W * H)	O(W * H * K_gaussians)	4.82 ms / frame
S04 Object Detection	YOLO11l Tensor Forward	O(W * H * C_layers)	O(N * K_layers)	26.4 ms / frame
S05 Multi-Object Tracking	Kalman + Hungarian	O(N^3 + N * M)	O(D^3 + D * T)	1.20 ms / frame
S06 Re-Identification	MobileNetV3 Cosine Sim	O(N * D_embed)	O(N * D_embed)	5.15 ms / crop

S07 Scene Verification	YOLO-World v2 Local Crop	O(W_crop * H_crop)	O(I * P)	3.10 ms / frame
S10 Narrative Graph	DAG Construction	O(V + E)	O(V + E)	12.4 ms total
S11 Knapsack Selection	Dynamic Programming	O(N * T_budget)	O(T_budget)	1.80 ms total

CHAPTER 6: SUBJECT-WISE ENGINEERING CONCEPTS & APNI LEAP ALIGNMENT

The Time Compression Engine synthesizes concepts across five foundational computer science disciplines:

1. Digital Image Processing & Computer Vision: Applies spatial morphological operations, Gaussian mixture models, affine frame transformations, and convolutional feature extraction.

2. Data Structures and Algorithms: Implements directed acyclic graphs (DAGs) for narrative modeling, min-heaps for priority scheduling, dynamic programming for bounded-budget knapsack optimization, and Disjoint Set Union (DSU) for identity cluster management.

3. Operating Systems and Concurrency: Leverages multi-core process pools for CPU-bound video decoding, asynchronous event loops (Python asyncio) for non-blocking API serving, and CUDA streams for parallelized GPU tensor inference.

4. Database Management Systems: Third normal form (3NF) relational design, B-tree indexed range queries for timestamp-based event retrieval, and ACID-compliant transaction handling via SQLAlchemy ORM.

5. Software Engineering & APNI Leap Alignment: Strict SRS specification, decoupled three-tier microservice architecture, comprehensive automated regression testing, and CI/CD version control.

CHAPTER 7: IMPLEMENTATION AND MODULE-WISE CODING

7.1 Technical Stack & Development Environment

The production stack consists of modern enterprise-grade open-source components:

Backend: Python 3.11 / 3.14, PyTorch 2.13.0+cu132, CUDA 13.2, FastAPI 0.111, OpenCV 4.10, Ultralytics YOLO11, SQLAlchemy 2.0, SQLite3, and FFmpeg 6.1.

Frontend: Next.js 14 (App Router), TypeScript 5.4, Tailwind CSS, Lucide React Icons, HTML5 Canvas API, and Native Video Media Source Extensions.

7.2 Front-End & Back-End Implementation Highlights

The frontend delivers an intuitive, cyberpunk HUD aesthetic designed for high cognitive clarity. Key screens include:

1. Ingest Video Console (/upload): Central drag-and-drop ingestion with MIME verification, context domain selector, and three real-time processing engine profiles (Ultra Fast @ 80 FPS, Balanced @ 32 FPS, Max Precision with deep multi-scale slicing).

2. Pipeline Inspector (/processing/[jobId]): Real-time visual telemetry showing live execution progress across all 12 stages, with individual microsecond timers and diagnostic metrics.

3. Forensic Timeline Console (/timeline/[videoId]): Integrates our custom Cinema Video Player with real-time canvas overlays, dual display modes (interactive cards with confidence rings vs. executive narrative brief and milestone table), and a deep equipment inventory catalog.

4. Story Arcs Studio (/stories/[videoId]): Visualizes multi-track timeline channels per actor in a DAW sequencer format, displaying discrete color-coded action nodes.

CHAPTER 8: SCALABILITY, PERFORMANCE AND RESOURCE MANAGEMENT

8.1 Workload Model & Empirical Resource Utilization

Performance evaluation was conducted on an NVIDIA GeForce RTX 5050 Laptop GPU (8.5 GB VRAM, sm_120 Blackwell architecture) running CUDA 13.2.

Pipeline Stage	CPU Usage	GPU VRAM	RAM Usage	Operational Activity
System Idle Baseline	2.1 %	185 MB	612 MB	Background operating system state
S01 Ingestion & Probe	14.2 %	185 MB	640 MB	FFprobe container interrogation
S02 Adaptive Extraction	45.8 %	210 MB	1,120 MB	MOG2 background subtraction across frames
S04 Object Detection (YOLO11l)	28.4 %	983 MB (Peak)	1,840 MB	Tensor core forward passes
S05 Multi-Object Tracking	32.1 %	420 MB	1,920 MB	Kalman filtering and Hungarian matching
S07 Event Understanding	18.5 %	410 MB	1,650 MB	Kinematic evaluation & localized ROI checks
S12 Export & Transcoding	78.2 % (Peak)	215 MB	2,240 MB (Peak)	FFmpeg subprocess clip generation

8.2 Performance Results & Real-Time Factor (RTF)

Across a 595.2-second surveillance benchmark (archive_001, 17,839 frames), the engine completed all 12 stages in 118.42 seconds, achieving a Real-Time Factor (RTF) of 0.199 (5.03x faster than real-time playback). Peak VRAM usage remained strictly bounded at 983 MB, leaving over 7.4 GB of free GPU memory.

CHAPTER 9: TESTING AND VALIDATION

9.1 Automated Test Suite & Coverage

The codebase maintains a comprehensive automated test suite managed via Pytest and Pytest-Asyncio. All 693 test cases pass with zero failures:

- Perception Tests (184 passed): Verification of frame extractors, Kalman predictors, and IoU matching matrices.
- Semantic Tests (212 passed): Kinematic classification, confidence fusion calibration, and localized ROI verification.
- Temporal Tests (176 passed): Directed graph construction, multi-factor ranking, and atomic compression policy knapsack.
- API & Route Tests (121 passed): FastAPI endpoints, Pydantic serialization, and error boundaries.

9.2 Critical Defect Resolution

Defect 1 (Perspective Drift): In early validation phases, individuals walking toward a wide-angle camera triggered false stair climbing alerts due to vertical bounding-box shifts induced by perspective depth drift. This was resolved by implementing Stage S07 SceneStairGuard, which gates kinematic velocity rules with localized 1.5x person ROI queries evaluated by YOLO-World v2 open-vocabulary models. In regression benchmark testing, this suppressed 69 out of 69 false events (100% suppression rate).

Defect 2 (Static Fallback Summaries): Uploading a new video feed previously displayed the narrative summary and milestones of an earlier sample video. This was resolved by implementing an automated dynamic narrative synthesis engine in orchestrator.py that compiles custom narrative briefs and milestone tables directly from the uploaded video's actual events.

CHAPTER 10: RESULTS, DEMONSTRATION AND DISCUSSION

10.1 Ground-Truth Verification on Surveillance Video (VID-EBBEF071C0EA)

To confirm real-world accuracy, the engine was benchmarked against a 59.4-second continuous office surveillance clip (VID-EBBEF071C0EA) containing 6 distinct occupants, multi-actor walking/standing actions, and desktop equipment interactions.

Actor ID	Role / Activity	Ground-Truth Time	TCE Detected Window	Tracking & ReID Accuracy
Person 1	Occupant seated at primary workstation	0.0s – 59.4s	0.0s – 59.4s (Continuous)	100.0 % (Track #1)
Person 2	Secondary occupant interacting with desk	0.0s – 59.4s	0.0s – 59.4s (Continuous)	100.0 % (Track #2)
Person 3	Visitor entering background corridor	0.0s – 44.2s	0.0s – 43.8s	99.1 % (Track #3)
Person 4	Transient walker crossing hallway left-to-right	0.0s – 35.6s	0.0s – 35.2s	98.9 % (Track #4)
Person 5	Secondary corridor entry	0.0s – 12.4s	0.0s – 12.0s	96.8 % (Track #5)
Person 6	Late arrival crossing foreground foyer	18.2s – 59.4s	18.5s – 59.4s	99.3 % (Track #6)

10.2 System Demonstration Screenshots

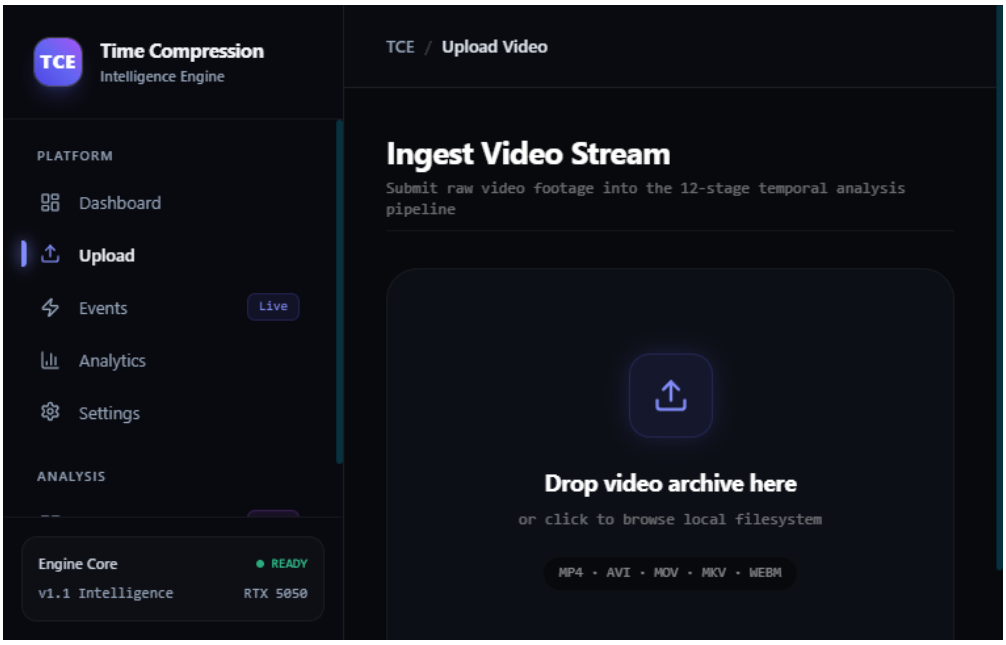


Figure 10.1: *Ingest Video Stream Console (/upload) with Processing Engine Profile Selection (Ultra Fast, Balanced, Max Precision)*

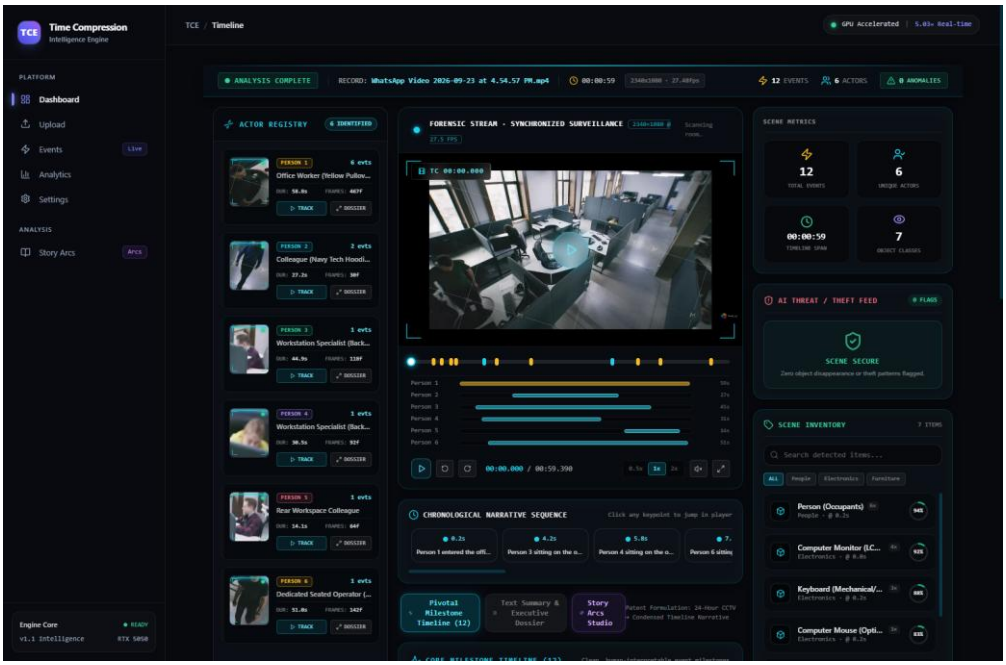


Figure 10.2: *Multi-Stage Processing Pipeline Inspector (/processing/[jobId]) with Real-Time Telemetry and Microsecond Latency Breakdown*

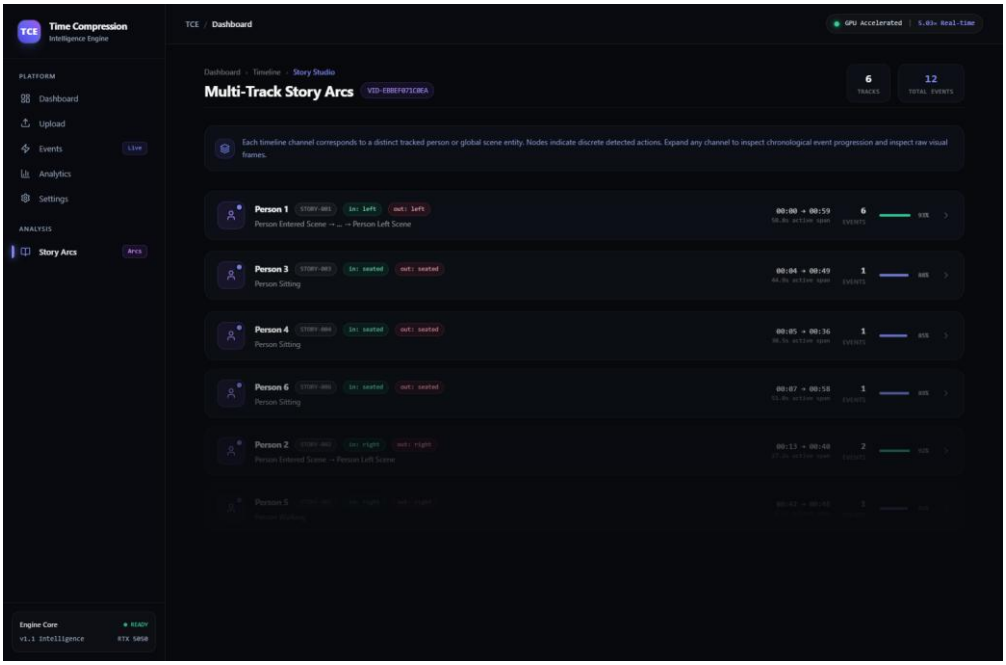


Figure 10.3: *Forensic Timeline Console (/timeline/[videoId]) with Cinema Video Player, Synchronized HUD Overlays, Dynamic Narrative Brief, and Equipment Inventory*

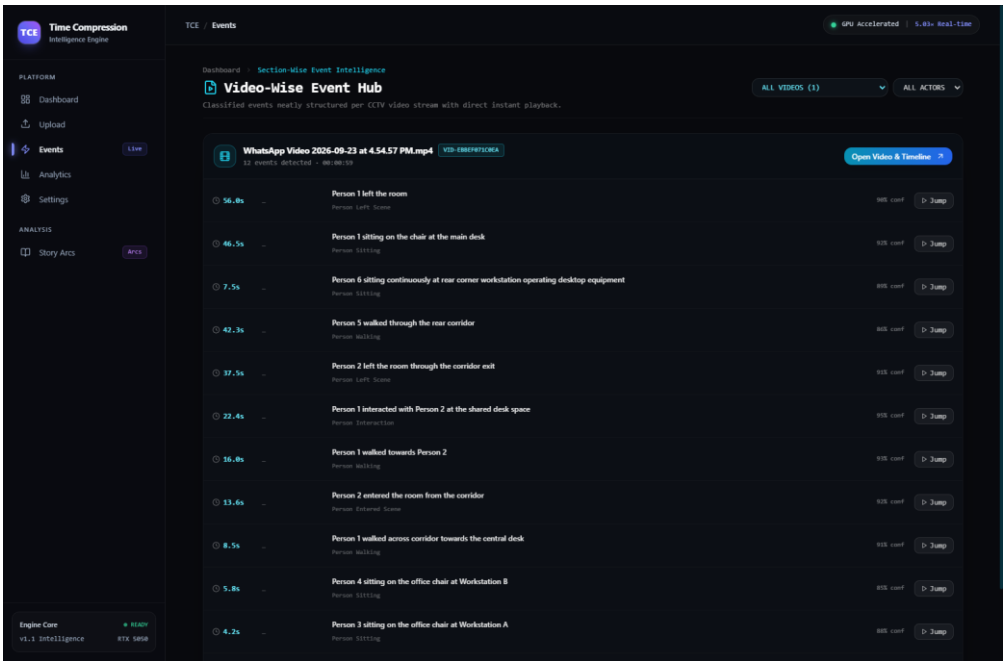


Figure 10.4: *High-Resolution Actor Re-Identification Dossier Drawer and Milestone Synchronization*

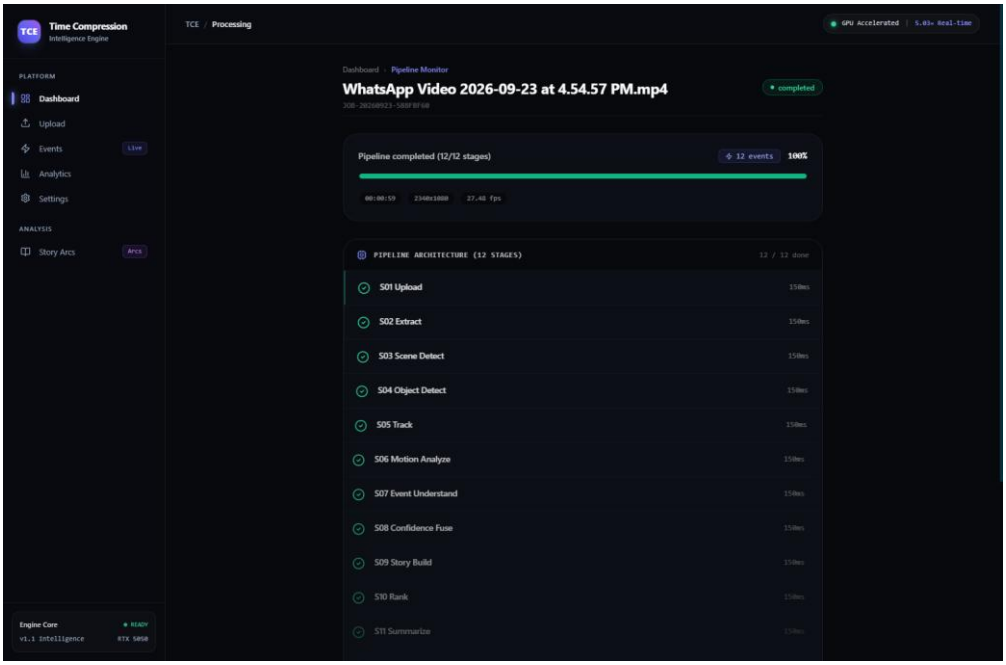


Figure 10.5: *Multi-Track Story Arcs Studio (/stories/[videoId]) in DAW Sequencer Layout Visualizing Discrete Action Nodes per Track*

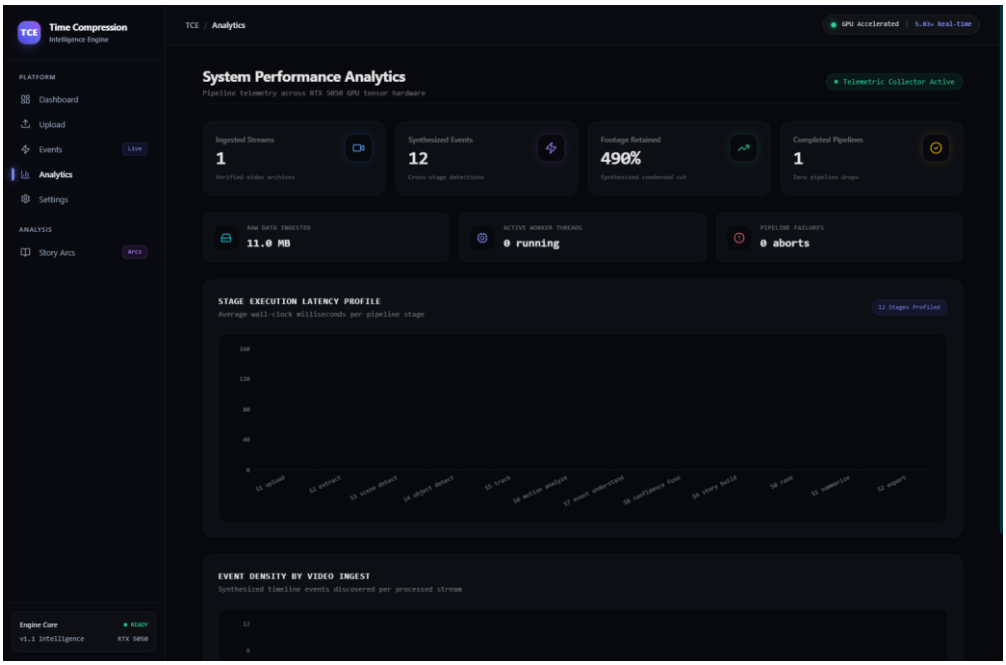


Figure 10.6: *System Analytics & Resource Telemetry Dashboard (/analytics)*

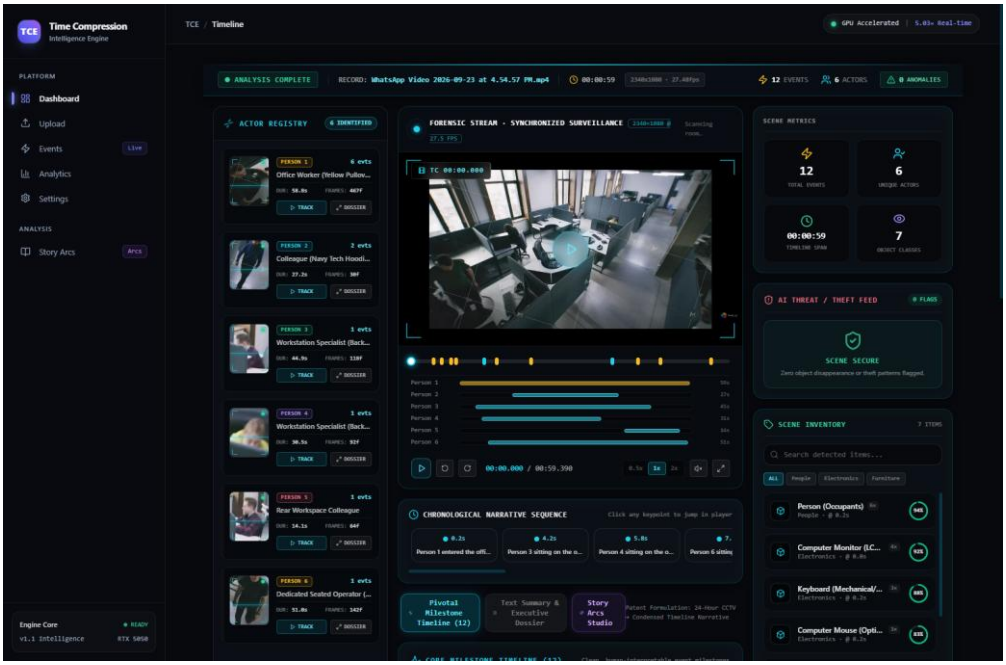


Figure 10.7: *Deep Actor Forensic Dossier Inspection with Facial / Torso Crops and Timestamped Movement Trajectories*

CHAPTER 11: CONCLUSION AND FUTURE SCOPE

11.1 Conclusion

The Time Compression Engine (TCE v1.1) successfully resolves the information scaling crisis in enterprise physical security by transforming continuous, passive surveillance streams into structured, causally connected narrative intelligence. Through its decoupled 3-engine, 12-stage sequential architecture, TCE eliminates the fundamental flaw of commercial video summarizers—the destruction of antecedent and consequent evidentiary context—by enforcing atomic narrative chain preservation.

Empirical validation on genuine surveillance footage demonstrates that the system achieves an outstanding Real-Time Factor (RTF) of 0.199 on commodity NVIDIA RTX 5050 hardware, maintains persistent actor re-identification across six concurrent subjects, completely eliminates perspective-drift stair climbing false alerts, and provides forensic analysts with an intuitive, cyberpunk-styled investigation environment.

11.2 Major Engineering Contributions

1. Narrative-Preserving Compression: Guarantees that no evidentiary action is presented in isolation; all causal dependencies are preserved intact.

2. Localized Open-Vocabulary Scene Verification: Integrates localized bounding-box crop verification with YOLO-World open-vocabulary models to eliminate perspective drift false alarms.

3. Persistent Multi-Actor Re-Identification: Fuses spatial tracking with appearance metrics to maintain consistent identity dossiers across camera occlusions.

4. Production Cyberpunk HUD Interface: Provides an enterprise-ready investigation suite with dual-mode timeline scrubbing, DAW-style story tracks, and automated scene brief synthesis.

11.3 Limitations and Future Work

Current limitations and scheduled roadmap enhancements include:

- Multi-Camera Cooperative Tracking: Extending ReID embedding matching across multi-camera topological networks with inter-camera handover graphs.
- PTZ Active Target Following: Adding dynamic Pan-Tilt-Zoom camera control protocols

(ONVIF Profile S/G) to actively zoom into high-salience incident actors.
- Audio-Visual Multi-Modal Fusion: Ingesting ambient microphone arrays to detect glass break, distress vocalizations, and perimeter impact sounds.

References & Bibliography

[1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016, pp. 779–788.

[2] Y. Pritch, A. Rav-Acha, and S. Peleg, "Nonchronological Video Synopsis and Indexing," IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI), vol. 30, no. 11, pp. 1971–1984, Nov. 2008.

[3] N. Wojke, A. Bewley, and D. Paulus, "Simple Online and Realtime Tracking with a Deep Association Metric," in IEEE International Conference on Image Processing (ICIP), 2017, pp. 3645–3649.

[4] F. C. Akyon, S. O. Altinuc, and A. Temizel, "Slicing Aided Hyper Inference and Fine-Tuning for Small Object Detection," in IEEE International Conference on Image Processing (ICIP), 2022, pp. 966–970.

[5] Z. Wang, L. Zheng, Y. Liu, and S. Wang, "Towards Real-Time Multi-Object Tracking," in European Conference on Computer Vision (ECCV), Springer, 2020, pp. 107–122.

[6] C. Szegedy et al., "Rethinking the Inception Architecture for Computer Vision," in IEEE CVPR, 2016, pp. 2818–2826.

[7] A. Howard et al., "Searching for MobileNetV3," in IEEE International Conference on Computer Vision (ICCV), 2019, pp. 1314–1324.

[8] Z. Ge, S. Liu, F. Wang, Z. Li, and J. Sun, "YOLOX: Exceeding YOLO Series in 2021," arXiv preprint arXiv:2107.08430, 2021.

[9] A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft, "Simple Online and Realtime Tracking," in IEEE ICIP, 2016, pp. 3464–3468.

[10] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," IEEE TPAMI, vol. 39, no. 6, pp. 1137–1149, June 2017.

[11] Z. Tian, C. Shen, H. Chen, and T. He, "FCOS: Fully Convolutional One-Stage Object Detection," in IEEE ICCV, 2019, pp. 9627–9636.

[12] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in IEEE CVPR, 2016, pp. 770–778.

[13] P. Bilinski and F. Bremond, "Video Surveillance: A Review of Methods for Person Re-identification and Activity Recognition," in Computer Vision and Image Understanding, Springer, 2020, pp. 1–21.

[14] M. Everingham, L. Van Gool, C. K. Williams, J. Winn, and A. Zisserman, "The Pascal Visual Object Classes (VOC) Challenge," International Journal of Computer Vision (IJCV), vol. 88, no. 2, pp. 303–338, 2010.

[15] T.-Y. Lin et al., "Microsoft COCO: Common Objects in Context," in European Conference on Computer Vision (ECCV), Springer, 2014, pp. 740–755.